

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence Application Development Practical
Practical – VI

Roll No.:T031	Name:Ishita Patel
Class:TYIT	Batch:2
Date of Assignment: 19/08/26	Date/Time of Submission: 19/08/26

1. Using a simple example (Burglary–Alarm), create a Bayesian Network using pgmpy.

Code:-

```
# Bayesian Network using pgmpy
# Example: Burglary-Alarm Network
```

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
```

```
# Step 1: Create Bayesian Network structure
```

```
model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
    ("Earthquake", "Alarm"),
    ("Alarm", "JohnCalls"),
    ("Alarm", "MaryCalls")
])
```

```
# Step 2: Define Conditional Probability Tables
```

```
# P(Burglary)
```

```
cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.999], [0.001]],
    state_names={
        "Burglary": ["False", "True"]
    }
)
```

```
# P(Earthquake)
```

```
cpd_earthquake = TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[[0.998], [0.002]],
    state_names={
        "Earthquake": ["False", "True"]
    }
)
```

```
# P(Alarm | Burglary, Earthquake)
```

```
cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.999, 0.71, 0.06, 0.05], # Alarm = False
```

```

    [0.001, 0.29, 0.94, 0.95] # Alarm = True
],
evidence=["Burglary", "Earthquake"],
evidence_card=[2, 2],
state_names={
    "Alarm": ["False", "True"],
    "Burglary": ["False", "True"],
    "Earthquake": ["False", "True"]
}
)

# P(JohnCalls | Alarm)
cpd_john = TabularCPD(
    variable="JohnCalls",
    variable_card=2,
    values=[
        [0.95, 0.10], # JohnCalls = False
        [0.05, 0.90] # JohnCalls = True
    ],
    evidence=["Alarm"],
    evidence_card=[2],
    state_names={
        "JohnCalls": ["False", "True"],
        "Alarm": ["False", "True"]
    }
)

# P(MaryCalls | Alarm)
cpd_mary = TabularCPD(
    variable="MaryCalls",
    variable_card=2,
    values=[
        [0.99, 0.30], # MaryCalls = False
        [0.01, 0.70] # MaryCalls = True
    ],
    evidence=["Alarm"],
    evidence_card=[2],
    state_names={
        "MaryCalls": ["False", "True"],
        "Alarm": ["False", "True"]
    }
)

# Step 3: Add CPDs to model
model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
    cpd_alarm,
    cpd_john,
    cpd_mary
)

# Step 4: Check model correctness
print("Is Bayesian Network valid?", model.check_model())

# Step 5: Perform inference

```

```
inference = VariableElimination(model)
```

```
# Query: Probability of Burglary when John and Mary both call
result = inference.query(
    variables=["Burglary"],
    evidence={
        "JohnCalls": "True",
        "MaryCalls": "True"
    }
)
```

```
print("\nProbability of Burglary given JohnCalls=True and MaryCalls=True:")
print(result)
```

Output:-

```
Is Bayesian Network valid? True

Probability of Burglary given JohnCalls=True and MaryCalls=True:
+-----+-----+
| Burglary | phi(Burglary) |
+=====+=====+
| Burglary(False) | 0.7158 |
+-----+-----+
| Burglary(True) | 0.2842 |
+-----+-----+
```

With another example:-

Code:-

```
# Bayesian Network using pgmpy
# Example: Smart Home Security Network
```

```
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
```

```
# Step 1: Create Bayesian Network structure
```

```
model = DiscreteBayesianNetwork([
    ("HomeAlone", "MotionDetected"),
    ("MotionDetected", "Alarm"),
    ("DoorOpen", "Alarm"),
    ("Alarm", "SecurityAlert")
])
```

```
# Step 2: Define Conditional Probability Tables
```

```
# P(HomeAlone)
```

```
cpd_home = TabularCPD(
    variable="HomeAlone",
    variable_card=2,
    values=[
        [0.40], # HomeAlone = False
        [0.60] # HomeAlone = True
    ],
    state_names={
        "HomeAlone": ["False", "True"]
    }
)
```

```

    }
)

# P(DoorOpen)

cpd_door = TabularCPD(
    variable="DoorOpen",
    variable_card=2,
    values=[
        [0.85], # DoorOpen = False
        [0.15] # DoorOpen = True
    ],
    state_names={
        "DoorOpen": ["False", "True"]
    }
)

# P(MotionDetected | HomeAlone)

cpd_motion = TabularCPD(
    variable="MotionDetected",
    variable_card=2,
    values=[
        [0.90, 0.30], # MotionDetected = False
        [0.10, 0.70] # MotionDetected = True
    ],
    evidence=["HomeAlone"],
    evidence_card=[2],
    state_names={
        "MotionDetected": ["False", "True"],
        "HomeAlone": ["False", "True"]
    }
)

# P(Alarm | MotionDetected, DoorOpen)

cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.99, 0.60, 0.40, 0.05], # Alarm = False
        [0.01, 0.40, 0.60, 0.95] # Alarm = True
    ],
    evidence=["MotionDetected", "DoorOpen"],
    evidence_card=[2, 2],
    state_names={
        "Alarm": ["False", "True"],
        "MotionDetected": ["False", "True"],
        "DoorOpen": ["False", "True"]
    }
)

# P(SecurityAlert | Alarm)

cpd_alert = TabularCPD(
    variable="SecurityAlert",

```

```

variable_card=2,
values=[
    [0.95, 0.10], # SecurityAlert = False
    [0.05, 0.90]  # SecurityAlert = True
],
evidence=["Alarm"],
evidence_card=[2],
state_names={
    "SecurityAlert": ["False", "True"],
    "Alarm": ["False", "True"]
}
)

# Step 3: Add CPDs to the model

model.add_cpds(
    cpd_home,
    cpd_door,
    cpd_motion,
    cpd_alarm,
    cpd_alert
)

# Step 4: Check model correctness

print("Is Bayesian Network valid?", model.check_model())

# Step 5: Perform inference

inference = VariableElimination(model)

# Query:
# Probability of HomeAlone when
# SecurityAlert=True and DoorOpen=True

result = inference.query(
    variables=["HomeAlone"],
    evidence={
        "SecurityAlert": "True",
        "DoorOpen": "True"
    }
)

# Display result

print("\nProbability of HomeAlone given SecurityAlert=True and DoorOpen=True:")
print(result)

```

Output:-

```

Is Bayesian Network valid? True

Probability of HomeAlone given SecurityAlert=True and DoorOpen=True:
+-----+-----+
| HomeAlone | phi(HomeAlone) |
+=====+=====+
| HomeAlone(False) | 0.2887 |
+-----+-----+
| HomeAlone(True) | 0.7113 |
+-----+-----+

```

2. Write a program to compute Posterior Probability using Bayesian Networks(Burglar|Alarm = True)

Code:-

Posterior Probability using Bayesian Network

Query: $P(\text{Burglary} \mid \text{Alarm} = \text{True})$

```
from pgmpy.models import DiscreteBayesianNetwork
```

```
from pgmpy.factors.discrete import TabularCPD
```

```
from pgmpy.inference import VariableElimination
```

Step 1: Create Bayesian Network structure

```
model = DiscreteBayesianNetwork([
    ("Burglary", "Alarm"),
    ("Earthquake", "Alarm")
])
```

Step 2: Define probability of Burglary

```
cpd_burglary = TabularCPD(
    variable="Burglary",
    variable_card=2,
    values=[[0.999], [0.001]],
    state_names={"Burglary": ["False", "True"]}
)
```

Step 3: Define probability of Earthquake

```
cpd_earthquake = TabularCPD(
    variable="Earthquake",
    variable_card=2,
    values=[[0.998], [0.002]],
    state_names={"Earthquake": ["False", "True"]}
)
```

Step 4: Define probability of Alarm

$P(\text{Alarm} \mid \text{Burglary}, \text{Earthquake})$

```
cpd_alarm = TabularCPD(
    variable="Alarm",
    variable_card=2,
    values=[
        [0.999, 0.71, 0.06, 0.05], # Alarm = False
        [0.001, 0.29, 0.94, 0.95] # Alarm = True
    ],
    evidence=["Burglary", "Earthquake"],
    evidence_card=[2, 2],
    state_names={
        "Alarm": ["False", "True"],
        "Burglary": ["False", "True"],
        "Earthquake": ["False", "True"]
    }
)
```

Step 5: Add CPDs to model

```
model.add_cpds(
    cpd_burglary,
    cpd_earthquake,
```

```

    cpd_alarm
)

# Step 6: Check whether model is correct
print("Is Bayesian Network valid?", model.check_model())

# Step 7: Perform inference
inference = VariableElimination(model)

result = inference.query(
    variables=["Burglary"],
    evidence={"Alarm": "True"}
)

print("\nPosterior Probability:")
print("P(Burglary | Alarm = True)")
print(result)

```

Output:-

```

Is Bayesian Network valid? True

Posterior Probability:
P(Burglary | Alarm = True)
+-----+-----+
| Burglary | phi(Burglary) |
+=====+=====+
| Burglary(False) | 0.6264 |
+-----+-----+
| Burglary(True) | 0.3736 |
+-----+-----+

```

With another example:-

Code:-

```

# Posterior Probability using Bayesian Network
# Query: P(Rain | DeliveryLate = True)

from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

# Step 1: Create Bayesian Network structure
model = DiscreteBayesianNetwork([
    ("Rain", "DeliveryLate"),
    ("Traffic", "DeliveryLate")
])

# Step 2: Define probability of Rain
cpd_rain = TabularCPD(
    variable="Rain",
    variable_card=2,
    values=[[0.75], [0.25]],
    state_names={"Rain": ["False", "True"]}
)

```

Step 3: Define probability of Traffic

```
cpd_traffic = TabularCPD(  
    variable="Traffic",  
    variable_card=2,  
    values=[[0.60], [0.40]],  
    state_names={"Traffic": ["False", "True"]}  
)
```

Step 4: Define probability of DeliveryLate

P(DeliveryLate | Rain, Traffic)

```
cpd_delivery = TabularCPD(  
    variable="DeliveryLate",  
    variable_card=2,  
    values=[  
        [0.95, 0.70, 0.60, 0.10], # DeliveryLate = False  
        [0.05, 0.30, 0.40, 0.90]  # DeliveryLate = True  
    ],  
    evidence=["Rain", "Traffic"],  
    evidence_card=[2, 2],  
    state_names={  
        "DeliveryLate": ["False", "True"],  
        "Rain": ["False", "True"],  
        "Traffic": ["False", "True"]  
    }  
)
```

Step 5: Add CPDs to model

```
model.add_cpds(  
    cpd_rain,  
    cpd_traffic,  
    cpd_delivery  
)
```

Step 6: Check whether model is correct

```
print("Is Bayesian Network valid?", model.check_model())
```

Step 7: Perform inference

```
inference = VariableElimination(model)
```

```
result = inference.query(  
    variables=["Rain"],  
    evidence={"DeliveryLate": "True"}  
)
```

```
print("\nPosterior Probability:")  
print("P(Rain | DeliveryLate = True)")  
print(result)
```

output:-

Is Bayesian Network valid? True

Posterior Probability:

$P(\text{Rain} \mid \text{DeliveryLate} = \text{True})$

Rain	phi(Rain)
------	-----------

Rain(False)	0.4286
-------------	--------

Rain(True)	0.5714
------------	--------